SYNTHETIC AUTOSAR-INSPIRED HLD

Electric Vehicle Powertrain Control - High-Level Design

Document ID: DEMO-EV-HLD-001 | Revision: 1.0 | Date: 15 October 2026

Classification: PUBLIC SYNTHETIC TEST DATA - NOT AN OEM SPECIFICATION

1. Purpose and system overview

This document describes a fictional AUTOSAR Classic Platform-inspired electric vehicle powertrain control architecture. It is intended solely to test an AI document assistant: PDF upload, semantic Q&A, component extraction, dependency visualization, consistency checks, and report export. The names and values below are invented for demonstration.

The application software receives vehicle speed, brake requests, battery limits, and diagnostic commands. The motor controller determines torque permission based on the latest valid input signals. The architecture uses sender-receiver communication for cyclic data and a client-server operation for diagnostic status retrieval.

2. Architecture boundaries

In scope: application software components, logical ports, interfaces, data elements, functional flow, and integration checks. Out of scope: physical CAN frame IDs, ECU pinout, memory allocation, real safety qualification, cybersecurity assessment, production timing validation, and complete AUTOSAR ARXML definitions.

3. Software component inventory

SWC ID	Component	Responsibility
SWC-01	VehicleSpeedSWC	Publishes validated vehicle speed.
SWC-02	BrakeControlSWC	Converts pedal input into a brake torque-reduction request.
SWC-03	BatteryManagementSWC	Publishes battery state of charge and allowable discharge power.
SWC-04	MotorControlSWC	Computes torque permission from speed, brake, and battery limits.
SWC-05	DiagnosticManagerSWC	Provides diagnostic status and stores application fault indicators.

All five SWCs are logical demonstration components; no claim is made that they are generated from real AUTOSAR tooling.

COMPONENT DETAILS AND PORTS

4. Software component descriptions

4.1 VehicleSpeedSWC (SWC-01): reads a validated wheel-speed estimate and publishes VehicleSpeedKph once every 10 ms. If the sensor estimate is invalid, it sets SpeedValid to false. Provides port PpVehicleSpeed typed by IF_VehicleSpeed.

4.2 BrakeControlSWC (SWC-02): reads the brake pedal percentage and publishes BrakeTorqueReductionPct once every 10 ms. Provides port PpBrakeRequest typed by IF_BrakeRequest. Requires port RpVehicleSpeed typed by IF_VehicleSpeed to support speed-dependent plausibility checking.

4.3 BatteryManagementSWC (SWC-03): publishes BatterySocPct and MaxDischargePowerKw once every 100 ms. Provides port PpBatteryStatus typed by IF_BatteryStatus.

4.4 MotorControlSWC (SWC-04): requires RpVehicleSpeed (IF_VehicleSpeed), RpBrakeRequest (IF_BrakeRequest), and RpBatteryStatus (IF_BatteryStatus). Publishes AllowedMotorTorqueNm using provided port PpMotorCommand typed by IF_MotorCommand. A missing valid brake request causes torque permission to fall to zero.

4.5 DiagnosticManagerSWC (SWC-05): provides client-server port PpDiagnosticStatus typed by IF_DiagnosticStatus. The GetDiagnosticStatus operation returns ActiveFaultCount and LastFaultCode. MotorControlSWC requires client port RpDiagnosticStatus to request the current diagnostic status during a maintenance diagnostic session.

5. Component ports and interfaces

Component	Provided ports	Required ports
VehicleSpeedSWC	PpVehicleSpeed / IF_VehicleSpeed	None
BrakeControlSWC	PpBrakeRequest / IF_BrakeRequest	RpVehicleSpeed / IF_VehicleSpeed
BatteryManagementSWC	PpBatteryStatus / IF_BatteryStatus	None
MotorControlSWC	PpMotorCommand / IF_MotorCommand	RpVehicleSpeed / IF_VehicleSpeed; RpBrakeRequest / IF_BrakeRequest; RpBatteryStatus / IF_BatteryStatus; RpDiagnosticStatus / IF_DiagnosticStatus
DiagnosticManagerSWC	PpDiagnosticStatus / IF_DiagnosticStatus	None

INTERFACES AND SIGNAL DEFINITIONS

6. Logical interface catalogue

Interface	Type	Data elements / operation
IF_VehicleSpeed	Sender-Receiver	VehicleSpeedKph (float32); SpeedValid (boolean)
IF_BrakeRequest	Sender-Receiver	BrakeTorqueReductionPct (uint8)
IF_BatteryStatus	Sender-Receiver	BatterySocPct (uint8); MaxDischargePowerKw (float32)
IF_MotorCommand	Sender-Receiver	AllowedMotorTorqueNm (float32)
IF_DiagnosticStatus	Client-Server	GetDiagnosticStatus() -> ActiveFaultCount (uint8), LastFaultCode (uint16)

7. Signal ownership and consumption

Signal / data element	Producer	Consumer(s)	Period
VehicleSpeedKph	VehicleSpeedSWC	BrakeControlSWC; MotorControlSWC	10 ms
SpeedValid	VehicleSpeedSWC	BrakeControlSWC; MotorControlSWC	10 ms
BrakeTorqueReductionPct	BrakeControlSWC	MotorControlSWC	10 ms
BatterySocPct	BatteryManagementSWC	MotorControlSWC	100 ms
MaxDischargePowerKw	BatteryManagementSWC	MotorControlSWC	100 ms
AllowedMotorTorqueNm	MotorControlSWC	Torque actuator adapter (external boundary)	10 ms

The diagnostic status operation is synchronous client-server communication rather than a cyclic sender-receiver signal.

8. Units and valid ranges

VehicleSpeedKph: 0 to 180 km/h. BrakeTorqueReductionPct: 0 to 100 percent. BatterySocPct: 0 to 100 percent. MaxDischargePowerKw: 0 to 75 kW. AllowedMotorTorqueNm: 0 to 250 Nm. These limits are fictional test values.

DEPENDENCIES AND FUNCTIONAL FLOWS

9. Explicit component dependency list

Source component	Target component	Reason
VehicleSpeedSWC	BrakeControlSWC	Speed data for brake plausibility checking
VehicleSpeedSWC	MotorControlSWC	Speed data for torque calculation
BrakeControlSWC	MotorControlSWC	Brake torque-reduction request
BatteryManagementSWC	MotorControlSWC	Battery discharge-power limit and SoC
DiagnosticManagerSWC	MotorControlSWC	GetDiagnosticStatus service is invoked by MotorControlSWC

Dependency direction in the table is producer/provider -> consumer/client. For IF_DiagnosticStatus, DiagnosticManagerSWC is the server and MotorControlSWC is the client.

10. Normal propulsion flow

Step 1: VehicleSpeedSWC publishes VehicleSpeedKph and SpeedValid. Step 2: BrakeControlSWC publishes BrakeTorqueReductionPct. Step 3: BatteryManagementSWC publishes BatterySocPct and MaxDischargePowerKw. Step 4: MotorControlSWC receives the latest valid values and calculates AllowedMotorTorqueNm. Step 5: MotorControlSWC publishes AllowedMotorTorqueNm to the external torque actuator adapter.

11. Fault response flow

If SpeedValid is false, MotorControlSWC uses a conservative torque limit of 40 Nm. If the brake request is missing or invalid, MotorControlSWC sets AllowedMotorTorqueNm to 0 Nm. If MaxDischargePowerKw is missing, MotorControlSWC also sets AllowedMotorTorqueNm to 0 Nm. These are demonstration behaviors, not production safety requirements.

12. Diagnostics flow

During a maintenance diagnostic session, MotorControlSWC invokes GetDiagnosticStatus on DiagnosticManagerSWC. DiagnosticManagerSWC returns ActiveFaultCount and LastFaultCode. The service call must not directly change AllowedMotorTorqueNm.

INTEGRATION REQUIREMENTS

13. Integration requirements

Requirement ID	Requirement text
REQ-001	MotorControlSWC shall read IF_VehicleSpeed, IF_BrakeRequest, and IF_BatteryStatus before calculating torque permission.
REQ-002	MotorControlSWC shall publish AllowedMotorTorqueNm every 10 ms while the application is running.
REQ-003	A missing or invalid BrakeTorqueReductionPct shall result in AllowedMotorTorqueNm = 0 Nm.
REQ-004	A missing MaxDischargePowerKw shall result in AllowedMotorTorqueNm = 0 Nm.
REQ-005	DiagnosticManagerSWC shall provide the GetDiagnosticStatus operation through IF_DiagnosticStatus.
REQ-006	All software component interfaces shall have a documented provider and consumer or an explicit external boundary.

14. Verification considerations

TEST-001: Verify VehicleSpeedKph is consumed by both BrakeControlSWC and MotorControlSWC. TEST-002: Verify an invalid brake request results in zero allowed torque. TEST-003: Verify missing battery power limit results in zero allowed torque. TEST-004: Verify the diagnostic client-server operation returns two values. TEST-005: Verify all documented dependency graph edges match Section 9.

15. Known limitations

This sample does not contain AUTOSAR ARXML, RunnableEntity mappings, RTE generation details, BSW configuration, MCAL configuration, real ECU hardware interfaces, network management, or a complete ISO 26262 safety case. It must not be used as a vehicle implementation specification.

REVISION REVIEW AND INTENTIONAL TEST FINDINGS

16. Revision metadata

Document ID: DEMO-EV-HLD-001. Revision: 1.0. Owner: Synthetic Test Engineering. Review status: DEMONSTRATION ONLY. Approval status: NOT APPROVED FOR PRODUCTION.

17. Intentionally inserted document inconsistencies

This section deliberately includes inconsistencies to test the AI assistant. The discrepancies are not real engineering defects and should be reported as potential review findings.

FINDING-SEED-A: Section 6 names the battery interface IF_BatteryStatus. An alternate integration note below incorrectly calls the same interface IF_BatteryState. The term IF_BatteryState is not defined in the interface catalogue.

Integration note INT-07: MotorControlSWC shall receive BatterySocPct from BatteryManagementSWC through IF_BatteryState.

FINDING-SEED-B: Section 4.2 specifies BrakeControlSWC publishes BrakeTorqueReductionPct every 10 ms. An outdated note below instead specifies a 20 ms publication period.

Outdated note INT-08: BrakeControlSWC shall publish BrakeTorqueReductionPct every 20 ms.

FINDING-SEED-C: Section 8 gives AllowedMotorTorqueNm a maximum of 250 Nm. The note below gives a different maximum of 300 Nm.

Outdated note INT-09: AllowedMotorTorqueNm has a valid range of 0 to 300 Nm.

18. Expected behavior of the document assistant

The assistant should extract five SWCs and five defined interfaces, recover the documented producer-to-consumer dependency relationships, cite relevant pages for answers, and flag the three contradictory or undefined statements in Section 17. An unanswerable question such as "What is the physical CAN ID for vehicle speed?" should be answered as not specified in this document.

End of synthetic test document.